

TARUMT Resorts Assignment Development Guideline

这份 guideline 是给团队开发用的。它不是最终报告，也不是完整答案。它的目标是让四个人知道：

- 现在项目已经确定什么。
- 每个 folder/package 应该放什么。
- 每个 module 只需要负责什么。
- 每个 module 的基本逻辑是什么。
- 每个人怎么分工，怎么对接。

0. 目前已经确定的东西

现在只确定两个大方向：

Architecture: ECB
Main ADT: AVL Tree / Search Tree

还没有确定的东西：

Queue / Stack / List / Hash Table 要不要用
每个 module 具体开几个 class
DAO 要不要现在做 file save/load
每个 report 要用什么排序方式

所以现在的 `assignment/` folder 只保留最小地基：

```
assignment/  
├── src/  
│   ├── main/  
│   ├── adt/  
│   ├── boundary/  
│   ├── control/  
│   ├── entity/  
│   ├── dao/  
│   └── utility/
```

`adt/` 现在只放 AVL main ADT 的最小 skeleton：

```
SearchTreeInterface.java  
AVLTree.java  
Visitor.java
```

1. 每个 package 的责任

`main`

只放程序入口。

应该做：

- 启动 main menu。
- 不写业务逻辑。
- 不直接处理 booking/member/room。

例子：

```
public class Main {  
    public static void main(String[] args) {  
        // later: new MainMenuControl().run();  
    }  
}
```

`adt`

放你们自己写的数据结构。

现已确定：

- `AVLTree`
- `SearchTreeInterface`
- `Visitor`

还没确定，之后有需要才加：

- Queue
- Stack
- List
- Hash Table
- Heap

规则：

- ADT 不应该知道 hotel/booking/member 是什么。
- ADT 不应该 `System.out.println()`。
- ADT 只负责资料结构操作，例如 insert/search/delete/traversal。

`entity`

放资料对象。

可能会有：

- `Booking`
- `Guest`
- `Room`
- `Member`
- `Payment`
- `RoomStatus`
- `VipKey`
- `PointsKey`

规则：

- Entity 只存资料。
- 可以有 constructor/getter/setter/toString。
- 不放 menu。
- 不放 Scanner。
- 不放 AVL rotation。

例子：

```
public class Member {  
    private String memberId;  
    private String name;  
    private int age;  
    private int points;  
}
```

`boundary`

放 Terminal UI。

应该做：

- show menu
- ask input
- display result

不应该做：

- AVL insert/delete
- calculate business rule
- save/load file

例子：

```
Boundary 问: Enter member id:  
Control 拿到 member id 后, 去 AVL search。  
Boundary 只负责显示 result。
```

`control`

放 module logic。

应该做：

- 调用 Boundary 拿 input。
- 创建 Entity object。
- 调用 AVL method。
- 决定业务流程。
- 调用 DAO save/load。

例子：

```
WalkInBookingControl.addBooking()  
1. ask guest details from UI  
2. create Booking object  
3. bookingTree.insert(confirmationNo, booking)  
4. show success message
```

`dao`

放资料来源和资料保存。

初期可以先不做 file，只做 seed data 或 shared data holder。

之后可能会有：

- `BookingDAO`
- `MemberDAO`

- `RoomDAO`
- `DataRepository`

规则：

- DAO 不负责 menu。
- DAO 不负责 AVL rotation。
- DAO 负责 load/save/seed。

`utility`

放共用 helper。

可能会有：

- `InputUtil`
- `DateUtil`
- `IdGenerator`
- `MessageUI`

规则：

- 只有很多 module 都会用，才放 utility。
- 单一 module 自己用的 helper，先放在自己的 control 里面也可以。

2. AVL 里面到底存什么？

AVL 存的是 key-value pair。

key = 用来比较大小和 search 的东西
value = 真正的 object

例子：

```
AVLTree<String, Member> memberTree = new AVLTree<>();  
memberTree.insert("M1001", new Member("M1001", "Ali", 20, 1200));
```

这里 AVL 比较的是 `"M1001"`，不是整个 `Member` object。

如果要用 object 当 key，这个 key class 必须可以比较：

```
public class VipKey implements Comparable<VipKey> {  
    public int compareTo(VipKey other) {  
        // decide priority order here  
    }  
}
```

3. SearchTreeInterface 现在为什么这么少？

现在 interface 只留最基础 method：

```
insert  
search  
delete  
contains  
traverseInOrder  
size  
isEmpty  
clear
```

原因：

- 你们还没决定每个 module 要什么特殊功能。
- 先不要把 `maxEntry`、`rangeSearch`、`removeMax` 这种 method 写死。
- 等 module 真的需要，再由 ADT 负责人加。

开发原则：

先用最小 interface 跑通。
module 有真实需求，再扩展 interface。

4. Module 1: Walk-In / Standard Booking

这个 module 要做什么？

负责普通客户或 walk-in 客户的 booking 流程。

可能功能：

- Add booking
- Search booking
- Cancel booking
- Update booking
- View booking list/report

主要资料

可能需要：

- `Booking`
- `Guest`
- `Room`

AVL 怎么用？

最自然的 key：

confirmationNo -> Booking

例子：

Key: B1001
Value: Booking object

基本逻辑

Add booking:

1. Boundary 问 guest name, IC/passport, date, room type
2. Control generate confirmation number
3. Control create Booking object
4. Control call bookingTree.insert(confirmationNo, booking)
5. Boundary display success message

Search booking:

1. Boundary 问 confirmation number
2. Control call bookingTree.search(confirmationNo)

3. 如果找到，Boundary display booking details
4. 如果找不到，Boundary display not found

Cancel booking:

1. search booking
2. 如果存在，把 booking status 改成 Cancelled
3. 选择做法 A: update same object
4. 选择做法 B: delete old record and insert updated record

现实 example

客人 Ali walk in，要订 Deluxe Room。

```
System create confirmationNo = B1001
Booking = Ali + Deluxe + 2026-07-10 + Pending
AVL insert:
B1001 -> Booking(Ali, Deluxe, Pending)
```

之后 front desk 要找 Ali 的 booking，只需要 search `B1001`。

这个 module 可能需要的辅助 ADT

还没决定。

如果你们之后要做 first come first served waiting line，可能需要 Queue。

如果只是普通 booking search/report，AVL 可以先够用。

5. Module 2: VIP Priority Room Allocation

这个 module 要做什么？

负责 VIP guest 的优先安排。

可能功能：

- Add VIP request
- Search VIP booking
- Allocate room to highest priority VIP
- View VIP waiting list
- Generate VIP report

主要资料

可能需要：

- `Booking`
- `Guest`
- `Room`
- `VipKey`

AVL 怎么用？

VIP 不适合只用 confirmation number 排序，因为 VIP 重点是 priority。

可能 key：

VipKey -> Booking

`VipKey` 可以包含：

vipTier
arrivalTime/order
confirmationNo

基本逻辑

Add VIP:

1. Boundary 问 VIP tier, guest details, room request
2. Control create Booking
3. Control create VipKey
4. vipTree.insert(vipKey, booking)

Allocate VIP:

1. 找出 priority 最高的 VIP
2. assign available room
3. update booking status
4. delete / mark allocated

注意：现在 `SearchTreeInterface` 还没有 `maxEntry()`。如果你们真的决定 VIP 要从最高 priority 取出，那 ADT 组之后可以加：

maxEntry()
deleteMax()
traverseReverseOrder()

现实 example

两个 VIP：

VIP Gold: arrival 10:00
VIP Platinum: arrival 10:10

虽然 Platinum 比 Gold 晚到，但 tier 更高，所以 priority 更高。

AVL key 不是 guest name，而是 `VipKey(tier, arrivalOrder)`。

这个 module 可能需要的辅助 ADT

还没决定。

如果只用 AVL + `VipKey`，已经可以做 priority ordering。

如果老师要求更像 priority queue，也可以讨论 Heap，但这会改变主 ADT 重点，所以要小心。

6. Module 3: Housekeeping / Room Status

这个 module 要做什么？

负责房间状态和清洁任务。

可能功能：

- Update room status
- Search room status

- View dirty rooms
- View ready rooms
- Assign housekeeping task
- Undo status update, if your team decides to support it

主要资料

可能需要：

- `Room`
- `RoomStatus`
- `HousekeepingTask`

AVL 怎么用？

最自然的 key：

roomNo -> RoomStatus

例子：

101 -> Dirty
102 -> Ready
103 -> Occupied

基本逻辑

Update room status:

1. Boundary 问 room number
2. Control search roomStatusTree.search(roomNo)
3. 如果 room 存在，修改 status
4. Control update AVL record
5. Boundary display updated status

View dirty rooms:

1. Control traverseInOrder(roomStatusTree)
2. 每个 roomStatus 检查 status == Dirty
3. Boundary display dirty room list

现实 example

Room 101 checkout 后变 Dirty。

roomStatusTree.search(101)
old status = Occupied
new status = Dirty
save back into AVL

Housekeeping 打扫完：

Room 101: Dirty -> Ready

这个 module 可能需要的辅助 ADT

还没决定。

如果你们要做 undo/redo，Stack 很适合。

如果只是 update/search/report , AVL 可以先够用。

7. Module 4: Front Desk / Service Counter

这个 module 要做什么？

负责 front desk 查询和服务。

可能功能：

- Search booking
- Check-in
- Check-out
- Search room availability
- Search guest/member
- Print service summary

主要资料

可能需要：

- `Booking`
- `Guest`
- `Room`
- `RoomStatus`
- `Member`

AVL 怎么用？

Front desk 通常会从多个 tree 查资料：

```
confirmationNo -> Booking  
roomNo -> Room  
memberId -> Member
```

基本逻辑

Check-in:

1. Boundary 问 confirmation number
2. Control search bookingTree
3. 如果 booking 存在 and status valid
4. assign/check room
5. update booking status = CheckedIn
6. update room status = Occupied

Check-out:

1. search booking
2. confirm guest checked in
3. update booking status = CheckedOut
4. update room status = Dirty

现实 example

Guest 给 confirmationNo `B1001`。

```
bookingTree.search("B1001")  
found Booking(Ali, Deluxe, Pending)  
status -> CheckedIn  
room 101 -> Occupied
```

这个 module 可能需要的辅助 ADT

还没决定。

如果 exact search 很多，可以讨论 Hash Table，但 AVL 的 $O(\log n)$ 已经可以接受，而且 AVL 更符合 main ADT。

8. Module 5: Loyalty / Rewards

这个 module 要做什么？

负责会员资料、积分、兑换。

可能功能：

- Add member
- Search member
- Update points
- Redeem points
- View member ranking/report

主要资料

可能需要：

- `Member`
- `Reward`
- `Transaction`

AVL 怎么用？

最基本 key：

memberId -> Member

例子：

M1001 -> Member(Ali, 1200 points)

基本逻辑

Add member:

1. Boundary 问 member details
2. Control create Member
3. memberTree.insert(memberId, member)

Add points:

1. search member by memberId
2. if found, member.points += earnedPoints
3. save/update member

Redeem:

1. search member
2. check enough points
3. deduct points
4. record redemption, if needed

现实 example

Ali check out 后赚 200 points。

```
memberTree.search("M1001")
old points = 1200
new points = 1400
```

这个 module 可能需要的辅助 ADT

还没决定。

如果要按 points 排名，可以讨论：

Option A: second AVL with PointsKey -> Member
Option B: traverse memberId tree, collect and sort

不要一开始就写死，等 report requirement 确认。

9. Report Module / Reports inside each module

要做什么？

大多数 assignment 会要求 reports。Report 可以放在每个 module 里面，也可以最后统一整理。

可能 report：

- Booking report
- VIP waiting/allocated report
- Room status report
- Member points report
- Daily check-in/check-out report

AVL 怎么用？

AVL 的优势是 sorted traversal。

```
traverseInOrder()
```

可以让资料按 key 顺序出来。

例子：

```
bookingTree.traverseInOrder(...)
```

会按 confirmation number 顺序拿到 booking。

如果 report 要按别的字段排序怎么办？

例如 member report 要按 points，不是 memberId。

选择还没定，之后讨论：

Option A: 再建一个 AVL , 用 PointsKey 当 key
Option B: traverse 后自己 sort
Option C: report 不排序 , 只 filter

原则 :

不要为了 report 乱加辅助 ADT。
先看 assignment 真正要求什么排序。

10. 四个人团队怎么分工

下面是一种推荐分法 , 不是强制。

Person A: ADT Lead

负责 :

- `SearchTreeInterface`
- `AVLTree`
- AVL Node
- height
- balance factor
- left rotate / right rotate
- insert
- search
- delete
- inorder traversal

产出 :

- 一个可以被其他 module 调用的 AVL。
- 写简单 test/demo , 让大家知道 method 怎么用。

Person A 不负责 :

- Walk-in booking business rule
- Front desk menu
- Loyalty points rule

Person B: Booking + Front Desk

负责 :

- Walk-In / Standard Booking
- Front Desk search/check-in/check-out
- `Booking`, `Guest`, maybe `Room`
- booking-related UI and Control

需要跟 Person A 对接 :

```
insert(confirmationNo, booking)
search(confirmationNo)
delete(confirmationNo)
traverseInOrder(visitor)
```

Person C: Room + VIP + Housekeeping

负责：

- Room status
- Housekeeping update
- VIP allocation logic
- `Room`, `RoomStatus`, maybe `VipKey`

需要跟 Person A 对接：

```
insert(roomNo, roomStatus)
search(roomNo)
traverseInOrder(visitor)
```

如果 VIP 真的需要最高 priority：

```
request Person A add maxEntry/deleteMax
```

不要自己偷偷在 module 里绕过 AVL。

Person D: Loyalty + Reports + Integration

负责：

- Member / Loyalty
- report formatting
- final menu integration
- shared utility / DAO coordination
- demo data

需要跟 Person A 对接：

```
insert(memberId, member)
search(memberId)
traverseInOrder(visitor)
```

如果 member ranking 要按 points：

```
和 team 讨论要不要 PointsKey second AVL
```

11. 四个人怎么配合开发

Step 1: 先定最小 shared contract

大家先同意 AVL 只有这些 method：

```
insert
search
delete
contains
traverseInOrder
size
isEmpty
clear
```

所有 module 先用这些 method 设计。

Step 2: 每个人先写自己的 Entity 草稿

例如：

```
Booking: confirmationNo, guestName, roomType, status
RoomStatus: roomNo, status
Member: memberId, name, points
```

先不要写太复杂。

Step 3: 每个 module 写自己的 Boundary + Control

每个 module 可以自己决定 file 名。

例子：

```
WalkInBookingUI.java
WalkInBookingControl.java
```

也可以：

```
BookingBoundary.java
BookingController.java
```

命名统一就好。

Step 4: 先用 fake data / simple object 测流程

ADT 还没完成时，module 同学可以先写 pseudocode 或 temporarily comment method call。

重点是先把流程想清楚：

```
input -> create object -> call AVL -> display result
```

Step 5: ADT 完成后开始 integration

integration 时检查：

- 有没有 package import 错。
- 有没有不同人重复写同一个 Entity。
- 有没有 Boundary 直接操作 AVL Node。
- 有没有 Entity 里面出现 Scanner/System.out。
- 有没有直接使用 Java Collections Framework，如果 assignment 禁止。

Step 6: 最后才做 polish

最后才做：

- nicer menu
- validation
- file save/load
- report formatting
- demo data
- presentation script

不要一开始就花太多时间美化 menu。

12. 每个 module 开发模板

每个人开发自己的 module 时，可以照这个模板写。

Module name:

1. Purpose

这个 module 解决什么问题？

2. Entity needed

需要什么资料 class？

3. AVL key

用什么 key search？

4. Main operations

Add / Search / Update / Delete / Report？

5. Flow

Boundary 问什么？

Control 做什么？

AVL 存什么？

6. Example

举一个真实 scenario。

7. Need extra ADT?

现在先写 No / Not yet。

真的需要时再讨论 Queue/Stack/List/Hash。

13. 什么情况下才加辅助 ADT

现在辅助 ADT 还没定。

判断规则：

业务天然需要某种行为，才加对应 ADT。

候选例子：

FIFO 排队 -> Queue

Undo / Redo -> Stack

Temporary report result -> List

Exact search average $O(1)$ -> Hash Table

Highest priority fast access -> Heap or AVL max method

注意：

- 不要为了看起来厉害而乱加。
- 加一个辅助 ADT，就要同步两份资料。
- 同步越多，bug 越多。
- 如果 AVL 已经够用，就先用 AVL。

14. 最重要的开发纪律

不要让 Boundary 变成大杂烩

Boundary 只做 input/output。

不好：

UI 里面直接写 AVL rotation

UI 里面直接 update booking status

好：

```
UI ask input
Control process
UI display result
```

不要让 Entity 做业务流程

Entity 是资料，不是 controller。

不好：

```
Booking.cancelBookingAndUpdateTree()
```

好：

```
Booking.setStatus("Cancelled")
BookingControl 决定什么时候 cancel
```

不要让 ADT 知道业务

AVLTree 不应该有：

```
insertBooking()
searchMember()
allocateVipRoom()
```

AVLTree 应该只有 generic method：

```
insert(K key, V value)
search(K key)
delete(K key)
```

15. Presentation 可以怎么讲

可以这样讲：

```
Our project follows ECB architecture.
Boundary handles terminal input and output.
Control handles business logic.
Entity stores data objects.
ADT contains our self-implemented AVL Tree.
```

```
We use AVL Tree as the main ADT because it keeps records searchable
and sorted with  $O(\log n)$  insert, search and delete.
```

```
Each module stores objects as values, and compares records using keys.
For example, booking confirmation number can be the key,
and the Booking object is the value.
```

```
Auxiliary ADTs are not fixed at the beginning.
We only add them if a module has a natural need,
such as Queue for FIFO or Stack for undo.
```

16. 下一步 checklist

Team discussion

- 确认有几个 module。
- 确认每个 module 的 required function。

- 确认每个 module 的 main AVL key。
- 确认是否真的需要辅助 ADT。

ADT work

- 完成 AVL insert/search/delete。
- 完成 rotation。
- 完成 inorder traversal。
- 写 demo 测试。

Module work

- 每个 module 先写 entity fields。
- 再写 boundary menu。
- 再写 control flow。
- 最后接 AVL。

Integration

- 所有人统一 package naming。
- 统一 main menu。
- 统一 error message style。
- 统一 sample data。
- 最后准备 presentation example。